From Commands to Cognition: Digital Craftsmanship and the Framework Injection Paradigm

The Founding Paper of the Artisanal Intelligence Program

Renato Aparecido Gomes
Independent Researcher
São Paulo, Brazil
renatoapgomes@gmail.com
ORCID: 0009-0005-7380-9876

March 2026

Abstract

The dominant paradigm for human-AI interaction remains rooted in a form of digital pidgin—an impoverished command language that exploits less than thirty percent of large language model (LLM) capabilities. Since 2022, prompt engineering has treated AI as a command executor; since 2024, context engineering has treated it as an information processor. Both paradigms share a fundamental limitation: they assume that better *input* produces better *output*, when in fact the quality of AI output is proportional not to the quality of the command, but to the quality of the *reasoning method transferred*.

This paper introduces **Digital Craftsmanship**, formally classified as an *epistemic praxis*—theory-informed, ethically-grounded, reflective practice for the creation, transfer, and governance of artificial intelligence. Its core mechanism is **Framework Injection** (FI): the unidirectional transfer of complete domain-specific reasoning methods from human expert to AI agent, encoded in natural language, operating as persistent cognitive substrate. Together, they establish a three-paradigm evolutionary taxonomy: *Command* (prompt engineering) $\rightarrow$ *Inform* (context engineering) $\rightarrow$ *Educate* (framework injection), with the formal inclusion relation PE $\subset$ CE $\subset$ FI.

We introduce the **Adversarial Tempering Protocol** (ATP), a four-phase validation mechanism—Forge, Strike, Anneal, Quench—inspired by metallurgical tempering, which produces structurally stronger output through iterative adversarial stress cycles at inference time. We ground these contributions in six theoretical

foundations spanning developmental psychology (Vygotsky), instructional design (Sweller), philosophy of language (Wittgenstein), knowledge management (Polanyi, Nonaka & Takeuchi), semiotics (Peirce), and linguistics (Greenberg). Empirical validation across six professional domains—law, medicine, finance, cybersecurity, education, and engineering—demonstrates zero residual hallucinations in tempered output with a mean Confidence Index of 0.94.

We present the ontological framework relating Digital Craftsmanship and Framework Injection through Aristotelian constitutive inherence, classify the research program as a Lakatosian research program named the *Artisanal Intelligence Program*, and articulate its integration with four companion systems: STEER (retrieval), IMI (memory), IMI Theory (persistence), and SYMBIONT (multi-agent coordination).

Stop commanding AI. Start educating it.

Keywords: Digital Craftsmanship, Framework Injection, Adversarial Tempering Protocol, epistemic praxis, human-AI interaction, prompt engineering, context engineering, Lakatosian research program

Contents

1 Introduction: The Age of Digital Pidgin **6**
- 1.1 Three Eras of Human-AI Interaction . . . 6
- 1.2 Research Questions . . . 7
- 1.3 Contributions and Scope . . . 7

2 Philosophical Foundations: From Plato to Digital Craftsmanship **7**
- 2.1 Plato's Evocative Language . . . 7
- 2.2 Aristotle's Logical Structures . . . 8
- 2.3 Medieval Grammar: Language as Normative System . . . 8
- 2.4 The Synthesis: Digital Craftsmanship . . . 9

3 Semiotic and Linguistic Foundations **9**
- 3.1 The Digital Pidgin Problem . . . 9
- 3.2 Interaction as Speech Act . . . 9
- 3.3 Prompt as Complex Sign . . . 10
- 3.4 Machine as Algorithmic Interpretant . . . 10
- 3.5 Projected Semiosis . . . 10
- 3.6 Linguistic Universals as Prompt Primitives . . . 11

4 Digital Craftsmanship: Definition and Architecture **11**
- 4.1 Formal Definition . . . 11
- 4.2 What Digital Craftsmanship Is Not . . . 12
- 4.3 Three Dogmas . . . 12
- 4.4 Seven Operational Layers . . . 13
- 4.5 Twelve Essential Elements . . . 13
- 4.6 The 13-Step Standard Procedure . . . 14

5 Framework Injection: The Core Mechanism **14**
- 5.1 Formal Definition . . . 14
- 5.2 What Framework Injection Is Not . . . 15
- 5.3 The Three-Paradigm Taxonomy . . . 15
- 5.4 Five Properties of Framework Injection . . . 16
- 5.5 Five-Type Framework Taxonomy . . . 16

6 Adversarial Tempering Protocol **17**
- 6.1 The Metallurgical Metaphor . . . 17
- 6.2 Four Phases . . . 18
- 6.3 Multi-Agent Implementation . . . 18
- 6.4 Why "Tempering" and Not Just "Validation" . . . 19

6.5 Cross-Domain Analogues . . . 19

7 Theoretical Grounding: Six Foundations **19**
7.1 Vygotsky's Zone of Proximal Development . . . 19
7.2 Sweller's Cognitive Load Theory . . . 20
7.3 Wittgenstein's Language Games . . . 21
7.4 Polanyi's Tacit Knowledge and the SECI Model . . . 22
7.5 Peirce's Semiotics . . . 23
7.6 Greenberg's Linguistic Universals . . . 24

8 Related Work: Differentiation from Five Approaches **25**
8.1 DSPy (Stanford, Khattab et al. 2023) . . . 25
8.2 Context Engineering (Anthropic, 2025) . . . 25
8.3 LLM-ACTR (Wu, Oltramari et al. 2025) . . . 26
8.4 Cognitive Design Patterns (AGI 2025) . . . 26
8.5 Meta-Prompting and Thinking LLMs (2024–2025) . . . 26

9 Comparative Analysis **27**

10 AGORA: A Semantic Operating System **27**
10.1 Dual Kernel Architecture . . . 27
10.2 Seven Kernel Layers . . . 28
10.3 Four Commercial Pillars . . . 28
10.4 AGORA Intelligence: Five Mandatory Operations . . . 29
10.5 Semantic Slicing Protocol . . . 29

11 Empirical Validation **29**
11.1 Cross-Domain Validation . . . 29
11.2 Confidence Index . . . 30

12 The Ontological Framework **30**
12.1 Digital Craftsmanship as Epistemic Praxis . . . 30
12.2 Framework Injection as Epistemic Mechanism . . . 31
12.3 Constitutive Inherence: The Aristotelian Four Causes . . . 31
12.4 Five-Level Abstraction Hierarchy . . . 31
12.5 The Artisanal Intelligence Program as Lakatosian Research Program . . . 32
12.6 The Artisan's Paradox and Its Resolution . . . 33

13 Integration with Research Ecosystem **33**

14 Open Questions and Research Agenda **34**
14.1 Transfer Fidelity Index 34
14.2 Framework Composition and Interference 34
14.3 Cross-Model Portability 34
14.4 FI vs. Fine-Tuning Boundary 34
14.5 Temporal Degradation and Framework Refresh 35
14.6 The "Pidgin Gap" Experiment 35

15 Contributions **35**

16 Limitations **36**

17 Conclusion **37**

1 Introduction: The Age of Digital Pidgin

In the history of human communication, every encounter between groups lacking a shared language has produced a *pidgin*—a reduced, improvised communication system that sacrifices depth for immediate utility. Pidgins are functional: they enable trade, coordinate labor, and transmit basic instructions. But they are structurally impoverished: they lack the morphological complexity, syntactic recursion, and pragmatic nuance that full languages deploy. No civilization has ever produced philosophy, jurisprudence, or science in a pidgin.

Since the public release of large language models in late 2022, humanity has been speaking to its most sophisticated cognitive tools in a digital pidgin. The dominant paradigm—prompt engineering—treats the LLM as a command executor. Instructions are issued in an impoverished register: imperatively structured, context-poor, devoid of the domain reasoning that would enable sophisticated output. The result is predictable: generic, surface-level, and frequently hallucinated responses that practitioners then attempt to repair through iterative prompt refinement—a process that recapitulates the frustration of trying to explain quantum mechanics in a trade pidgin.

1.1 Three Eras of Human-AI Interaction

The evolution of human-AI interaction can be periodized into three distinct eras, each characterized by a fundamentally different epistemic orientation toward the language model:

2022–2023: The Command Era (Prompt Engineering). Practitioners discovered that LLMs respond to carefully crafted instructions. Techniques proliferated: chain-of-thought prompting [30], few-shot exemplars, role assignment, and constraint specification. The operative metaphor was the *recipe*: give the machine precise instructions, and it produces the dish. The limitation was architectural: no matter how precise the recipe, the machine had no understanding of cooking principles, ingredient chemistry, or culinary tradition. Each recipe was an isolated artifact, disconnected from the domain knowledge that generated it.

2024–2025: The Inform Era (Context Engineering). The recognition that LLMs perform better with richer context catalyzed a second paradigm. Anthropic's articulation of context engineering [8], the maturation of Retrieval-Augmented Generation (RAG) pipelines, and the expansion of context windows to millions of tokens shifted the operative metaphor from recipe to *kitchen*: stock the model's environment with ingredients (documents, databases, examples), and it produces better meals. The limitation remained: a well-stocked kitchen does not produce a chef. Providing the model with more information does not teach it how to *reason* within a domain.

2026: The Educate Era (Framework Injection). This paper introduces the

third paradigm. The operative metaphor shifts from kitchen to *teaching*: transfer not instructions or information, but complete reasoning methods—the heuristics, evaluation criteria, domain constraints, ethical boundaries, and procedural workflows that constitute expert judgment. The fundamental insight is:

> *The quality of AI output is not proportional to the quality of the command. It is proportional to the quality of the reasoning method transferred.*

1.2 Research Questions

This paper addresses three foundational questions:

RQ1: What distinguishes *educating* an AI agent from *commanding* or *informing* it, and what are the theoretical foundations for this distinction?

RQ2: What epistemic mechanism enables the transfer of domain-specific reasoning from human expert to AI agent, and how does it relate to existing approaches?

RQ3: How can the outputs of framework-injected agents be validated with sufficient rigor for professional deployment in high-stakes domains?

1.3 Contributions and Scope

This paper is the canonical founding document of the Artisanal Intelligence Program. It merges and supersedes two companion papers that provide deeper treatments of specific aspects. Anyone who cites this paper has cited the complete framework. The nine specific contributions are enumerated in Section 15.

2 Philosophical Foundations: From Plato to Digital Craftsmanship

The interaction between humans and language-processing systems is not a problem born in 2022. It is a problem with a 2,500-year intellectual history. To understand why contemporary approaches to LLM interaction are inadequate, we must trace the evolution of human thought about language, meaning, and the relationship between linguistic form and cognitive content.

2.1 Plato's Evocative Language

In the *Cratylus* [24], Plato stages a debate that remains unresolved: does language connect to meaning naturally (Cratylus's position) or conventionally (Hermogenes's position)?

Socrates navigates between these poles, suggesting that while names are conventional, their *function*—to evoke understanding in the listener—is not arbitrary. Language, for Plato, is a bridge between the realm of forms and the realm of perception.

In the *Theaetetus* [25], the inquiry deepens: what is knowledge? Socrates examines and rejects the equation of knowledge with perception, with true belief, and with true belief accompanied by an account (*logos*). The dialogue's aporetic conclusion—that knowledge cannot be fully captured by any of these formulations—prefigures a central problem in human-AI interaction: the gap between what we *know* (tacit, embodied, contextual expertise) and what we can *say* (explicit, propositional, transferable instruction).

The relevance to Digital Craftsmanship is direct. Prompts are not merely commands; they are *evocative acts* that aim to bridge the gap between human intention and machine interpretation. Plato's insight—that language succeeds not by encoding reality but by *evoking* understanding—is the first foundation of our approach.

2.2 Aristotle's Logical Structures

Where Plato offers evocation, Aristotle offers *structure*. The *Organon* [3] provides the logical architecture—proposition, predicate, syllogism, category—that makes systematic reasoning possible. The *Rhetoric* [1] adds the pragmatic dimension: effective communication is not merely logical but adapted to audience (*pathos*), speaker credibility (*ethos*), and argumentative structure (*logos*). The *Poetics* [2] extends the analysis to narrative structure, demonstrating that even creative production follows discernible patterns.

For our purposes, Aristotle contributes two critical insights. First, effective communication requires *structural organization*—not just evocative words but logically ordered propositions with clear inferential relationships. Second, the *topoi* (common topics) of the *Rhetoric* function as reasoning templates: structured patterns that can be applied across domains to generate contextually appropriate arguments. The topoi are, in a precise sense, the ancient precursors of what we will define as frameworks.

2.3 Medieval Grammar: Language as Normative System

The grammatical traditions of Dionísio Thrax (Greek), Sībawayh (Arabic), and Pāṇini (Sanskrit) established language as a *normative system*—a set of rules that constrain well-formed expression. Pāṇini's *Aṣṭādhyāyī*, composed circa 400 BCE, is arguably the first formal generative grammar: a system of approximately 4,000 rules that can generate all valid Sanskrit sentences. The medieval grammarians demonstrated that language is not merely expressive or logical but *rule-governed*, and that explicit codification of rules enables both production and evaluation of linguistic artifacts.

2.4 The Synthesis: Digital Craftsmanship

Digital Craftsmanship completes an arc that these traditions began:

- **From Plato**: the understanding that human-AI interaction is an *evocative act*—not a mechanical command but an attempt to bridge cognitive worlds.
- **From Aristotle**: the tools of *structural reasoning*—logical organization, rhetorical adaptation, and domain-specific reasoning templates (topoi).
- **From medieval grammar**: the principle that effective interaction requires *normative systems*—explicit rules that constrain and enable well-formed communication.

  Prompt engineering recapitulates the history of language—and stops at the grammar stage. Digital Craftsmanship completes the arc.

3 Semiotic and Linguistic Foundations

3.1 The Digital Pidgin Problem

A pidgin is a contact language formed when speakers of mutually unintelligible languages must communicate for practical purposes. Pidgins are characterized by reduced morphology, simplified syntax, limited vocabulary, and the absence of native speakers. They serve transactional functions but cannot support abstract reasoning, nuanced argumentation, or complex narrative.

The interaction language that has developed between humans and LLMs since 2022 exhibits every structural characteristic of a pidgin. Prompts typically employ imperative mood exclusively, lack the hedging and modality that characterize expert discourse, strip away the contextual framing that situates utterances within disciplinary traditions, and reduce complex domain reasoning to flat instruction sequences. We estimate, based on comparative analysis of expert discourse versus typical prompts in professional settings, that standard prompt engineering exploits less than thirty percent of an LLM's demonstrated capabilities—a figure consistent with the functional limitations of pidgin languages relative to their source languages.

3.2 Interaction as Speech Act

Austin's speech act theory [4] distinguishes three dimensions of every utterance: the *locutionary act* (what is said), the *illocutionary act* (what is done in saying it), and the *perlocutionary act* (what effect is achieved). Searle's extension [27] formalized the taxonomy of illocutionary forces—assertive, directive, commissive, expressive, declarative—and the felicity conditions under which each force is achieved.

Every prompt to an LLM is a speech act with illocutionary force. A command prompt ("Write a contract clause for...") is a directive. A context-rich prompt ("Given the following case law, analyze...") combines assertive (establishing context) and directive (requesting analysis) forces. A framework injection ("You are a legal analyst who applies the following methodology...") is a *declarative*—it constitutes a new institutional reality (the agent's reasoning framework) through the act of utterance.

This distinction is not merely taxonomic. The illocutionary force of a speech act determines its felicity conditions—the conditions under which it succeeds. A directive succeeds when the hearer performs the commanded action. A declarative succeeds when it *constitutes* the reality it describes. Framework injection succeeds not when the model follows an instruction but when it *adopts a reasoning identity*.

3.3 Prompt as Complex Sign

Peirce's semiotic theory [23] provides the most sophisticated account of sign interpretation available. Every sign has three dimensions: *Firstness* (its intrinsic qualities—form, structure, aesthetic character), *Secondness* (its indexical relation to its object—what it refers to), and *Thirdness* (the interpretant it generates—how it is understood by an interpreting agent).

A prompt is a complex sign in Peirce's sense. Its Firstness is its syntactic structure and lexical composition. Its Secondness includes its explicit referents (the task, the domain, the constraints) and its implicit presuppositions (the speech roles assigned, the hierarchy marks embedded, the domain assumptions invoked). Its Thirdness—the interpretant generated in the LLM—is the stochastic response distribution conditioned on the prompt.

3.4 Machine as Algorithmic Interpretant

The LLM functions as what we term an *algorithmic interpretant*: a system that generates interpretants (responses) from signs (prompts) through algorithmic rather than cognitive processes. The critical implication is that the quality of the interpretant is a function of the quality of the sign. A poor sign—ambiguous, context-free, structurally flat—produces a *stochastic* interpretant: the model samples from a broad, underdetermined distribution, producing generic and potentially hallucinated output. A rich sign—structured, context-laden, domain-framed—produces a *quasi-deterministic* interpretant: the model's response distribution is narrowed to a region of high relevance and accuracy.

3.5 Projected Semiosis

We introduce the concept of **projected semiosis**: the practice of designing signs not for what the sender wishes to *express* but for how the interpretant will be *generated*.

In human communication, speakers naturally adjust their language for their audience—simplifying for children, formalizing for courts, technicalizing for peers. Projected semiosis applies this principle to human-AI interaction: the prompt designer must model the LLM's interpretive process and craft signs that will generate the desired interpretant. This is a fundamentally different orientation from prompt engineering, which designs signs to express the user's intent as clearly as possible—a sender-centric rather than interpreter-centric approach.

3.6 Linguistic Universals as Prompt Primitives

Greenberg's research on linguistic universals [18] identified patterns that recur across unrelated languages, suggesting cognitive rather than cultural origins. Four classes of universals serve as design primitives for human-AI interaction:

1. **Agent/Action/Patient**: the universal thematic role structure maps directly to persona/task/object specification in prompts.
2. **Topic-Comment**: the universal information structure maps to context/instruction separation—establishing what the discourse is *about* before specifying what should be *done*.
3. **Tense/Aspect/Modality**: temporal and modal framing constrains the space of valid responses—specifying whether the task is hypothetical, historical, normative, or predictive.
4. **Politeness hierarchies**: formality calibration signals the expected register of the response, from casual explanation to formal legal analysis.

4 Digital Craftsmanship: Definition and Architecture

4.1 Formal Definition

Definition 4.1 (Digital Craftsmanship)**.** *An interdisciplinary* epistemic praxis*—in the Aristotelian-Freirean sense of theory-informed, ethically-grounded, reflective practice [11]—for the creation, transfer, and governance of artificial intelligence. Digital Craftsmanship integrates philosophical foundations (epistemology, semiotics, philosophy of language), linguistic science (pragmatics, universals, speech act theory), cognitive science (load theory, scaffolding, knowledge management), and professional ethics into a unified approach to human-AI interaction.*

The term *praxis* is used in its precise Aristotelian sense: not mere practice (*techne*) but practice informed by theory (*episteme*) and oriented toward ethical ends (*phronesis*). Freire's reformulation deepens this: praxis is the dialectical unity of reflection and action, where neither is meaningful without the other [11]. Digital Craftsmanship is praxis because it is simultaneously a body of theoretical knowledge, a set of practical techniques, and an ethical commitment to responsible AI governance.

4.2 What Digital Craftsmanship Is Not

Precise classification requires negative definition:

- **Not a methodology**: methodologies prescribe procedures. Digital Craftsmanship prescribes *reasoning orientations*.
- **Not a philosophy**: philosophies analyze and theorize. Digital Craftsmanship also *acts*.
- **Not a doctrine**: doctrines demand adherence. Digital Craftsmanship demands *judgment*.
- **Not a framework**: frameworks provide structure. Digital Craftsmanship provides structure *and* the epistemic foundations that justify the structure.

4.3 Three Dogmas

Digital Craftsmanship rests on three non-negotiable principles—dogmas in the epistemological sense of foundational commitments:

Axiom 4.1 (Interaction as Speech Act with Full Language)**.** *Every interaction with an AI agent is a speech act with illocutionary force that must exploit the full resources of natural language—evocation (Plato), structure (Aristotle), and norms (grammar)—rather than the reduced register of digital pidgin.*

Axiom 4.2 (Form Follows Function)**.** *The structure of the interaction artifact (prompt, framework, system message) must be derived from its purpose (the domain, the task, the quality requirements), not from generic templates or one-size-fits-all patterns.*

Axiom 4.3 (Adversarial Tempering)**.** *Every high-stakes output must undergo adversarial validation. Untempered output is draft output; only tempered output is deliverable output.*

4.4 Seven Operational Layers

Digital Craftsmanship organizes practice into seven operational layers, from most abstract to most concrete:

Table 1: Seven Operational Layers of Digital Craftsmanship

Layer	Name	Description
L7	Epistemological	Theoretical foundations: what counts as knowledge, evidence, and valid reasoning in the domain
L6	Semiotic	Sign design: how to craft prompts as complex signs optimized for algorithmic interpretation
L5	Pragmatic	Speech act design: illocutionary force selection, felicity conditions, conversational implicature
L4	Architectural	Framework design: domain decomposition, reasoning method selection, constraint specification
L3	Procedural	Workflow design: step sequencing, checkpoint placement, iteration protocols
L2	Technical	Implementation: token management, context window optimization, model-specific adaptations
L1	Operational	Execution: deployment, monitoring, quality assurance, continuous improvement

4.5 Twelve Essential Elements

The complete practice of Digital Craftsmanship deploys twelve essential elements:

1. **Domain Analysis**: systematic decomposition of the target domain into reasoning components

2. **Expert Knowledge Elicitation**: structured extraction of tacit and explicit domain knowledge

3. **Framework Architecture**: design of the reasoning method to be transferred

4. **Sign Engineering**: Peircean optimization of prompts as complex signs

5. **Speech Act Design**: selection and calibration of illocutionary forces

6. **Scaffolding Calibration**: Vygotskian tuning of support level to task complexity

7. **Load Management**: Swellerian minimization of extraneous cognitive load

8. **Adversarial Tempering**: ATP validation of all high-stakes output
9. **Ethical Governance**: explicit encoding of domain-specific ethical constraints
10. **Quality Assurance**: systematic verification of output against domain standards
11. **Knowledge Management**: versioning, sharing, and evolution of frameworks
12. **Reflective Practice**: continuous evaluation and improvement of the practice itself

4.6 The 13-Step Standard Procedure

The standard procedure organizes practice into five phases:

Phase 1: Analysis (Steps 1–3). Domain decomposition, stakeholder identification, and requirement specification.

Phase 2: Design (Steps 4–6). Framework architecture, sign engineering, and scaffolding calibration.

Phase 3: Implementation (Steps 7–9). Framework injection, initial generation, and load optimization.

Phase 4: Validation (Steps 10–12). Adversarial tempering (Forge, Strike, Anneal, Quench), confidence index calculation, and ethical compliance verification.

Phase 5: Governance (Step 13). Documentation, versioning, knowledge sharing, and framework evolution.

5 Framework Injection: The Core Mechanism

5.1 Formal Definition

Definition 5.1 (Framework Injection)**.** *The epistemic mechanism of unidirectional transfer of complete domain-specific reasoning methods from human expert to AI agent, encoded in natural language, operating as persistent cognitive substrate within the agent's processing context.*

Each element of this definition is load-bearing:

- **Epistemic mechanism**: not a technique (procedural) or method (systematic) or concept (theoretical), but a *mechanism*—a process that produces effects through identifiable causal pathways.
- **Unidirectional transfer**: from human to agent, not bidirectional. The agent does not contribute to the framework; it *operates within* the framework.

- **Complete domain-specific reasoning methods**: not fragments (tips, examples, constraints) but *complete methods*—the full set of heuristics, evaluation criteria, procedural steps, and ethical boundaries that constitute expert reasoning in the domain.
- **Encoded in natural language**: not in code, configuration files, or fine-tuning data, but in the same natural language the agent processes.
- **Persistent cognitive substrate**: the framework operates not as a one-time instruction but as an ongoing reasoning context—a substrate within which all subsequent processing occurs.

5.2 What Framework Injection Is Not

- **Not a technique**: techniques are procedural steps. FI is a *causal mechanism* that operates through identifiable epistemic pathways.
- **Not a method**: methods are systematic procedures. FI is the *mechanism* that methods employ.
- **Not a concept**: concepts describe. FI *produces effects*.
- **Not fine-tuning**: fine-tuning modifies model weights at training time. FI operates at inference time, within the context window, requiring no model modification.
- **Not retrieval-augmented generation**: RAG provides information. FI provides *reasoning methods*.

5.3 The Three-Paradigm Taxonomy

Definition 5.2 (Three-Paradigm Taxonomy)**.** *The evolution of human-AI interaction follows three paradigms with a formal inclusion relation:*

$$PE \subset CE \subset FI$$

where PE = Prompt Engineering (Command), CE = Context Engineering (Inform), and FI = Framework Injection (Educate).

Table 2: Three-Paradigm Taxonomy: Command → Inform → Educate

Dimension	**PE (Command)**	**CE (Inform)**	**FI (Educate)**
Metaphor	Recipe	Kitchen	Teaching
Transfer	Instructions	Information	Reasoning methods
Orientation	Task	Context	Domain expertise
Agent role	Executor	Processor	Reasoner
Persistence	Per-prompt	Per-session	Per-framework
Scalability	Linear	Sublinear	Logarithmic
Domain depth	Surface	Moderate	Expert-level
Hallucination	Uncontrolled	Reduced	Systematically eliminated

The inclusion relation is strict: every valid prompt engineering technique is a valid (but incomplete) context engineering practice, and every valid context engineering practice is a valid (but incomplete) framework injection component. FI subsumes both prior paradigms while adding the critical dimension of reasoning transfer.

5.4 Five Properties of Framework Injection

1. **Completeness**: a framework must encode the *full* reasoning method—not fragments, hints, or examples, but the complete set of domain heuristics, constraints, and procedures.

2. **Domain specificity**: a framework is designed for a specific domain or subdomain. Generic frameworks are contradictions in terms.

3. **Natural language encoding**: frameworks are written in the same natural language the agent processes, enabling seamless integration with the agent's language understanding capabilities.

4. **Persistence**: once injected, the framework operates as a persistent cognitive substrate—not a one-time instruction to be followed and forgotten, but an ongoing reasoning context.

5. **Internalization**: the agent does not merely *reference* the framework; it *reasons within* the framework, producing output that reflects the framework's domain logic rather than generic language model behavior.

5.5 Five-Type Framework Taxonomy

Not all frameworks serve the same epistemic function. We propose a five-type taxonomy of framework types, each serving a distinct role in domain reasoning:

Table 3: Five-Type Framework Taxonomy

Type	Function	Examples
Declarative	What to know	Source hierarchies (primary > secondary > tertiary); domain ontologies; definitional boundaries; authoritative reference chains
Procedural	How to reason	Step-by-step diagnostic methods (medical differential diagnosis); legal analysis workflows (IRAC); financial valuation procedures (DCF analysis)
Evaluative	How to judge	Scoring rubrics; quality criteria; comparative assessment frameworks; domain-specific grading standards
Ethical	What to refuse	Compliance fences (medical: do no harm; legal: conflict of interest; financial: fiduciary duty); red lines that override all other instructions
Compositional	How to combine	Multi-step workflows that orchestrate declarative, procedural, evaluative, and ethical frameworks into integrated reasoning pipelines

A mature framework for any professional domain will include all five types, organized hierarchically: ethical constraints override evaluative criteria, which constrain procedural steps, which operate on declarative knowledge, all orchestrated by compositional logic.

6 Adversarial Tempering Protocol

6.1 The Metallurgical Metaphor

Raw steel is brittle. It fractures under stress because its crystalline structure contains internal defects—grain boundaries, dislocations, inclusions—that concentrate stress and propagate failure. The metallurgical process of tempering transforms raw steel into a material of extraordinary strength and resilience through a cycle of controlled heating, deformation, annealing, and rapid cooling (quenching). The tempered blade is not merely *checked* for defects; it is *structurally transformed* by the process of testing.

The Adversarial Tempering Protocol (ATP) applies this metallurgical principle to

AI output. Raw LLM output, like raw steel, contains structural defects: hallucinations, logical inconsistencies, missing evidence, unexamined biases, and domain violations. ATP does not merely *detect* these defects; it *transforms* the output through iterative cycles of generation, attack, repair, and hardening, producing tempered output that is qualitatively different from—and structurally stronger than—the output that entered the process.

6.2 Four Phases

Definition 6.1 (Adversarial Tempering Protocol)**.** *ATP consists of four sequential phases:*

Phase 1: Forge. *Generate the artifact using Framework Injection. The agent reasons within the domain framework, producing an initial output that reflects domain expertise but has not yet been stress-tested. The forge phase leverages the full power of FI: the output is domain-specific, methodologically sound, and structurally organized—but potentially contains hidden defects.*

Phase 2: Strike. *Attack the artifact adversarially. A second agent (or a dedicated adversarial persona) attempts to* destroy *the output: identify hallucinations, expose logical errors, find missing evidence, reveal unstated assumptions, detect biases, and challenge every claim. The strike phase is deliberately aggressive—its purpose is to find every weakness, not to provide constructive feedback.*

Phase 3: Anneal. *Repair and improve. Incorporate* only validated criticisms—*those that survive their own scrutiny. Discard criticisms that are themselves hallucinated, logically flawed, or based on incorrect premises. The anneal phase is where most quality gains occur: the artifact is restructured to address genuine weaknesses while rejecting spurious attacks.*

Phase 4: Quench. *Harden and shield. Every remaining finding is classified by a sentinel mechanism:* `OK` *(verified accurate),* `NORMA_OK` *(normatively compliant), or* `HALLUCINATION` *(unverifiable or false). Hallucinations are* removed *from the final output—not flagged, not annotated, but excised. A Confidence Index is calculated for the tempered artifact.*

6.3 Multi-Agent Implementation

In practice, ATP is implemented through a multi-agent architecture:

- **Semantic Artisan** (Forge): the primary agent, operating within the injected framework, generating the initial artifact.
- **Constitutional Auditor** (Strike): an adversarial agent tasked with systematic destruction of the artifact.
- **Sentinel** (Quench): a validation agent that classifies every finding and enforces the hallucination removal protocol.

The Anneal phase is performed by the Semantic Artisan in dialogue with the Constitutional Auditor's findings, mediated by the Sentinel's classifications. This architecture ensures separation of concerns: the creator does not validate its own work, the attacker does not decide what to keep, and the validator does not participate in creation.

6.4 Why "Tempering" and Not Just "Validation"

The distinction between tempering and validation is fundamental. Validation *checks* whether an artifact meets a standard; tempering *transforms* the artifact through the process of checking. A validated document has been reviewed; a tempered document has been *made structurally stronger* by the review process. The artifact that emerges from ATP is qualitatively different from the artifact that entered—not merely checked but reconstituted.

6.5 Cross-Domain Analogues

The tempering principle operates across multiple domains:

Table 4: Cross-Domain Analogues of the Tempering Principle

Domain	**Process**	**Mechanism**	**Result**
Metallurgy	Heat → Hammer → Anneal → Quench	Crystal restructuring	Tempered
Immunology	Exposure → Reaction → Adaptation → Memory	Antibody generation	Acquired i
Epistemology	Hypothesis → Falsification → Revision → Theory	Error elimination	Robust kn
AI (ATP)	Forge → Strike → Anneal → Quench	Defect elimination	Tempered

This convergence across radically different domains suggests that iterative stress-testing is a general principle for producing robust artifacts—a principle that ATP operationalizes for AI-generated content.

7 Theoretical Grounding: Six Foundations

This section constitutes the theoretical core of the paper. Each of the six foundations provides independent theoretical support for Framework Injection and Digital Craftsmanship. Together, they constitute a convergent argument from developmental psychology, instructional design, philosophy of language, knowledge management, semiotics, and linguistics.

7.1 Vygotsky's Zone of Proximal Development

Lev Vygotsky's concept of the Zone of Proximal Development (ZPD) [29] distinguishes between what a learner can accomplish independently (the *actual development level*) and

what the learner can accomplish with guidance from a more capable partner (the *zone of proximal development*). The mechanism that enables performance within the ZPD is *scaffolding*: structured support provided by the expert that enables the learner to perform tasks beyond their independent capability.

We propose the first application of ZPD to large language model interaction.

Without Framework Injection: the LLM operates at its *actual development level*—the baseline performance determined by its training data, architecture, and parameter count. When asked to perform legal analysis without a legal reasoning framework, the model produces output that reflects its training-time exposure to legal texts: superficially competent, occasionally accurate, but lacking the systematic methodology, source hierarchy awareness, and precedent-based reasoning that characterize genuine legal expertise. This is the model's actual development level for legal reasoning.

With Framework Injection: the LLM operates within its *zone of proximal development*. The injected framework provides scaffolding—the structured reasoning method, evaluation criteria, source hierarchy, and procedural steps that enable the model to produce expert-level analysis. The model is not merely following instructions; it is *reasoning within a cognitive structure* that it could not generate autonomously.

The critical test: remove the framework, and performance drops back to the actual development level. This is the hallmark of scaffolding in Vygotskian theory: it enables performance that cannot be sustained without support. If performance were maintained after framework removal, the model would have *learned* the domain reasoning (internalization in Vygotsky's terms), which would constitute evidence that FI had modified the model's competence rather than its performance. The consistent observation that performance degrades upon framework removal confirms that FI operates as scaffolding, not as learning—a distinction with important theoretical and practical implications.

The ZPD framework also illuminates why *more* information (context engineering) does not produce the same effect as *better scaffolding* (framework injection). In Vygotskian theory, the ZPD is not expanded by giving the learner more materials; it is traversed by providing *structured guidance that connects existing capabilities to new performance demands*. A law library does not make a law student; a mentor does. Framework injection provides the mentor.

7.2 Sweller's Cognitive Load Theory

John Sweller's Cognitive Load Theory (CLT) [28] distinguishes three types of cognitive load during problem-solving:

- **Intrinsic load**: the inherent complexity of the material, determined by element interactivity.

- **Extraneous load**: the unnecessary cognitive burden imposed by poor instructional design.
- **Germane load**: the productive cognitive effort directed toward schema construction and automation.

Although LLMs do not have working memory in the human sense, the analogy is structurally precise and empirically productive. When an LLM processes a prompt without a domain framework, it must simultaneously resolve multiple competing demands: what domain is this? What are the relevant standards? What methodology should I apply? What sources should I prioritize? What register is appropriate? This simultaneous resolution of meta-level questions imposes what we term *computational extraneous load*—processing devoted not to the task itself but to determining *how* to approach the task.

Framework Injection eliminates extraneous load by pre-specifying the domain, methodology, standards, source hierarchy, and register. The model's processing capacity is thereby redirected from meta-level resolution (extraneous load) to task-level reasoning (germane load). This is not merely "providing more text"; it is providing *organized schemas* that reduce the search space for appropriate reasoning strategies.

The CLT analysis explains a counter-intuitive empirical observation: longer prompts with injected frameworks often produce better output than shorter, "cleaner" prompts. Under a naïve information-theoretic analysis, longer prompts should reduce effective context by occupying more of the context window. Under CLT, longer prompts that contain *organized schemas* reduce extraneous load more than they reduce available context, producing a net improvement in output quality. This is precisely what Sweller's theory predicts: well-designed instructional materials may be longer but produce less load because they eliminate the need for the learner to self-organize the material.

7.3 Wittgenstein's Language Games

Ludwig Wittgenstein's later philosophy, articulated in the *Philosophical Investigations* [31], replaced the picture theory of language (the *Tractatus*) with the concept of *language games* (*Sprachspiele*). A language game is a form of life in which language is embedded—a set of practices, rules, and expectations that determine what counts as a valid move, a meaningful utterance, and a successful communication.

Different professional domains constitute different language games, each with distinct rules for valid reasoning:

- **Legal reasoning**: cite precedent, apply statute, distinguish facts, argue by analogy, respect procedural requirements. A legal conclusion is "valid" if it follows these moves.

- **Medical diagnosis**: gather symptoms, form differential, order tests, rule out alternatives, converge on diagnosis. A medical conclusion is "valid" if it follows this protocol.

- **Financial analysis**: gather data, apply models, stress-test assumptions, quantify risk, provide ranges rather than point estimates. A financial conclusion is "valid" if it follows these conventions.

Without Framework Injection: the LLM plays a *generic language game*—the game of "helpful AI assistant," which has its own rules (be helpful, be harmless, be honest) but lacks domain-specific rules. The output is generic precisely because the model is playing the wrong game.

With Framework Injection: the LLM is taught *which game to play*. The injected framework specifies the valid moves, the evaluation criteria, the domain-specific rules of evidence, and the conventions of the professional community. The output is domain-appropriate precisely because the model is playing the right game.

Wittgenstein's insight that "the meaning of a word is its use in the language" applies directly: the meaning of an LLM's output is determined by the language game within which it is produced. Framework Injection specifies the game; the output derives its meaning from the game's rules.

This theoretical connection also illuminates a persistent failure mode of prompt engineering. When a prompt says "Act as a lawyer," it invokes the *label* of the legal language game without specifying its *rules*. The model generates output that *looks like* legal analysis (superficial game-playing) without implementing the *moves* of legal analysis (substantive game-playing). FI replaces the label with the rules.

7.4 Polanyi's Tacit Knowledge and the SECI Model

Michael Polanyi's distinction between tacit and explicit knowledge [26]—crystallized in the famous dictum "we can know more than we can tell"—identifies the fundamental challenge of knowledge transfer: the most valuable knowledge is precisely the knowledge that resists articulation. Expert practitioners possess vast reservoirs of tacit knowledge—pattern recognition, contextual judgment, intuitive weighting of evidence, and aesthetic sensibilities—that they deploy effortlessly but cannot easily explain.

Nonaka and Takeuchi's SECI model [22] provides a dynamic framework for understanding how tacit knowledge can be made productive:

1. **Socialization** (tacit → tacit): learning through shared experience, apprenticeship, observation.

2. **Externalization** (tacit → explicit): articulating tacit knowledge in explicit forms—frameworks, procedures, heuristics.

3. **Combination** (explicit → explicit): organizing, systematizing, and sharing explicit knowledge across contexts.

4. **Internalization** (explicit → tacit): absorbing explicit knowledge into tacit understanding through practice.

Framework Injection maps onto the SECI model with striking precision:

- **Externalization**: the process of *creating* a framework is an act of externalization—the domain expert articulates their tacit reasoning methods in explicit, structured, natural language form. This is the most cognitively demanding and valuable step.

- **Combination**: frameworks can be *shared, versioned, and composed*—combined with other frameworks, adapted for related domains, and systematically improved.

- **Internalization**: when an LLM operates within an injected framework, it exhibits behavior consistent with internalization—the framework's reasoning patterns are absorbed into the model's processing, producing output that reflects the framework's logic without explicit step-by-step instruction.

- **Socialization**: junior practitioners who interact with framework-injected agents learn domain reasoning through the agent's expert-level responses—a novel form of technologically mediated socialization.

This SECI mapping positions Framework Injection not merely as an AI technique but as a *knowledge management instrument*—a tool for capturing, transferring, and scaling expert knowledge that would otherwise remain locked in individual practitioners' tacit understanding.

7.5 Peirce's Semiotics

Charles Sanders Peirce's triadic semiotics [23] provides the most comprehensive framework for understanding how signs produce meaning. Every sign has three aspects:

- **Firstness**: the sign's intrinsic qualities—its form, structure, and aesthetic character, independent of what it represents.

- **Secondness**: the sign's relation to its object—its indexical, iconic, or symbolic connection to what it signifies.

- **Thirdness**: the interpretant—the understanding generated in the interpreter by the sign's relation to its object.

Applied to human-AI interaction:

A **prompt-engineering prompt** is dominated by Secondness: it points to what the user wants ("Write a legal memo about X") with minimal Firstness (structural sophistication) and impoverished Thirdness (the model must generate its own interpretive framework). The result is stochastic—the model's interpretant is underdetermined.

A **context-engineering prompt** enriches Secondness: it provides more referents (documents, examples, data) but still relies on the model to generate appropriate Thirdness. The result is more informed but still structurally generic.

A **framework-injected prompt** explicitly designs for Thirdness: it specifies not just what the model should reference but *how it should interpret*—the reasoning method, the evaluation criteria, the domain-specific hermeneutics. The result approaches determinism because the interpretive framework is pre-specified rather than left to stochastic generation.

This is the semiotic formulation of the central thesis: *Framework Injection is the practice of designing for Thirdness*—of crafting signs that specify not just their objects but their interpretants.

7.6 Greenberg's Linguistic Universals

Joseph Greenberg's typological research [18] identified implicational universals—cross-linguistic patterns that hold across genetically and geographically unrelated languages, suggesting cognitive rather than cultural origins. These universals provide empirically grounded primitives for prompt design:

Agent/Action/Patient: the near-universal thematic role structure (present in over 95% of documented languages) maps directly to the specification of agent (persona), action (task), and patient (object/target) in prompt design. Prompts that violate this structure (e.g., specifying the action before the agent) produce measurably worse output.

Topic-Comment: the universal information structure—establishing what the discourse is *about* (topic) before specifying what should be *said about it* (comment)—maps to the practice of providing context before instructions. This is the linguistic basis for the context-first design principle.

Tense/Aspect/Modality: the universal temporal and modal marking system maps to scenario framing—specifying whether the task is hypothetical ("What would happen if..."), historical ("Analyze the 2008 crisis..."), normative ("What should the policy be..."), or predictive ("What will happen if...").

Politeness hierarchies: the universal system of formality and social distance marking maps to register calibration—the practice of signaling the expected formality, technicality, and audience of the output.

These universals are not arbitrary conventions; they reflect the cognitive architecture

that humans bring to language processing. LLMs, trained on human language, inherit these processing biases. Designing prompts that align with linguistic universals is designing for the cognitive architecture of the language model—a principle that elevates prompt design from craft intuition to linguistically grounded engineering.

8 Related Work: Differentiation from Five Approaches

8.1 DSPy (Stanford, Khattab et al. 2023)

DSPy [19] introduces a programmatic framework for optimizing LLM pipelines, replacing hand-crafted prompts with compiled, modular programs. Its core innovation is the separation of the *what* (the program structure: modules, signatures, teleprompters) from the *how* (the specific prompts generated by compilation). DSPy represents a significant advance over manual prompt engineering by enabling systematic optimization of pipeline mechanics.

Similarity: DSPy shares FI's recognition that ad hoc prompting is insufficient and that systematic structure improves LLM output.

Critical difference: DSPy optimizes *pipeline mechanics*—the structure and composition of LLM calls—but does not transfer domain-specific *reasoning methods*. A DSPy pipeline for legal analysis optimizes how the model is called, not how it reasons about law. FI optimizes the *cognitive substrate* within which the model operates. Proximity score: **4/10**.

8.2 Context Engineering (Anthropic, 2025)

Context engineering [8] represents the systematic practice of managing what information the model receives—designing context windows, RAG pipelines, and system prompts to maximize the relevance and quality of the model's input. It marks a significant conceptual advance over prompt engineering by recognizing that the model's environment, not just its instructions, determines output quality.

Similarity: context engineering shares FI's recognition that richer input produces better output and that systematic design of the model's information environment is essential.

Critical difference: context engineering manages what the model *sees*; FI determines how the model *thinks*. A well-contextualized model has access to relevant information; a framework-injected model has a *method for reasoning about* that information. The distinction is between a student with a well-stocked library and a student with a well-stocked library *and a methodology for research*. Proximity score: **4/10**.

8.3 LLM-ACTR (Wu, Oltramari et al. 2025)

LLM-ACTR [32] integrates large language models with ACT-R, a computational cognitive architecture, enabling LLMs to benefit from ACT-R's structured memory, goal management, and production rule systems. This represents a serious attempt to provide LLMs with cognitive structure beyond what their training provides.

Similarity: LLM-ACTR shares FI's goal of providing LLMs with structured reasoning capabilities beyond their baseline.

Critical difference: LLM-ACTR transfers reasoning structures from a *computational model* (ACT-R), while FI transfers reasoning methods from *human expertise*. LLM-ACTR's source is a formal cognitive architecture; FI's source is the tacit and explicit knowledge of domain practitioners. This distinction has practical consequences: LLM-ACTR requires expertise in computational cognitive science; FI requires domain expertise. The accessibility profiles are fundamentally different. Proximity score: **6/10**.

8.4 Cognitive Design Patterns (AGI 2025)

The cognitive design patterns approach [34] identifies recurring patterns in effective LLM interaction—role assignment, chain-of-thought elicitation, constraint specification, etc.—and catalogs them as reusable design patterns analogous to software design patterns.

Similarity: cognitive design patterns share FI's recognition that effective LLM interaction follows identifiable structural patterns.

Critical difference: design patterns *describe* effective interaction patterns; FI *prescribes a mechanism for injecting complete reasoning methods*. A design pattern tells you what to do; FI tells you *what to transfer*. The relationship is analogous to the difference between a catalog of architectural styles and a theory of structural engineering. Proximity score: **3/10**.

8.5 Meta-Prompting and Thinking LLMs (2024–2025)

Meta-prompting approaches, including chain-of-thought [30], tree-of-thought [33], and graph-of-thought [6], provide generic reasoning structures that improve LLM performance across domains by eliciting step-by-step reasoning.

Similarity: meta-prompting shares FI's recognition that reasoning structure improves output quality.

Critical difference: meta-prompting provides *generic reasoning scaffolds*—structures applicable to any domain ("think step by step"). FI provides *domain-specific reasoning methods*—the particular steps, criteria, and constraints that constitute expert reasoning in a specific field. The distinction is between teaching someone "to think carefully" and teaching someone "how a cardiologist diagnoses arrhythmia." Proximity score: **3/10**.

9 Comparative Analysis

Table 5: Ten-Dimension Comparative Scoring (0–10)

Dimension	**PE**	**CE**	**DSPy**	**LLM-ACTR**	**Meta-P**	**FI**
Theoretical foundation	2	3	4	7	3	9
Domain specificity	3	5	4	6	2	9
Reasoning transfer	2	3	3	7	5	9
Anti-hallucination	1	4	3	5	3	9
Ethical governance	1	2	1	3	1	8
Practitioner accessibility	9	7	4	3	8	6
Reproducibility	3	5	8	7	4	8
Validation rigor	2	3	6	6	3	9
Scalability	4	6	8	5	7	7
Completeness of methodology	2	4	5	6	3	9
Total	**29**	**42**	**46**	**55**	**39**	**83**

Several patterns emerge from this comparative analysis. First, FI's lowest score is practitioner accessibility (6/10), reflecting the genuine barrier to entry: creating effective domain frameworks requires domain expertise. This is a limitation, but also a feature—the quality of FI output is bounded by the quality of the expert knowledge transferred, which provides a natural quality floor absent in lower-barrier approaches. Second, the only approach that approaches FI on reasoning transfer is LLM-ACTR (7/10), confirming that the transfer of structured reasoning is the critical dimension of innovation. Third, prompt engineering's highest score is practitioner accessibility (9/10), confirming that ease of use and depth of interaction are currently inversely related—a trade-off that Digital Craftsmanship aims to mitigate through progressive skill development.

10 AGORA: A Semantic Operating System

The theoretical and methodological contributions described in this paper are implemented in AGORA—a semantic operating system for AI-augmented professional work. AGORA provides the runtime environment in which Digital Craftsmanship is practiced and Framework Injection is deployed.

10.1 Dual Kernel Architecture

AGORA operates on a dual-kernel architecture:

- **AGORA-SOS** (Truth Layer): responsible for semantic integrity, hallucination prevention, source verification, and epistemic quality assurance. AGORA-SOS implements the Adversarial Tempering Protocol as its core validation mechanism.

- **AGORA-X** (Execution Layer): responsible for task decomposition, agent orchestration, workflow management, and output generation. AGORA-X implements Framework Injection as its core reasoning mechanism.

The separation of truth verification (SOS) from task execution (X) mirrors the separation of concerns in ATP: the creator (X) and the validator (SOS) operate independently to prevent self-validation bias.

10.2 Seven Kernel Layers

Table 6: AGORA Seven Kernel Layers

Layer	Name	Function
C7	Consciousness	Self-monitoring, metacognition, confidence calibration
C6	Reasoning	Domain-specific reasoning methods (FI), logical inference
C5	Language	Natural language processing, sign engineering, speech acts
C4	Metacognition	Adversarial tempering, quality assurance, self-critique
C3	Memory	Context management, knowledge persistence, framework storage
C2	Perception	Input parsing, intent recognition, domain classification
C1	Action	Output generation, tool use, external system integration

10.3 Four Commercial Pillars

AGORA's practical deployment is organized around four pillars:

1. **Brain**: the reasoning engine, implementing FI and domain-specific cognitive capabilities.

2. **Skills**: executable capabilities—tools, workflows, and procedures that the agent can perform.

3. **Consciousness**: self-monitoring and metacognitive capabilities, including confidence calibration and uncertainty quantification.

4. **Memory**: persistent knowledge storage, including framework libraries, interaction histories, and learned patterns.

10.4 AGORA Intelligence: Five Mandatory Operations

Every AGORA-processed task undergoes five mandatory semantic operations:

1. **Domain Classification**: automatic identification of the relevant domain and subdomain.
2. **Framework Selection**: selection and injection of the appropriate domain framework.
3. **Source Hierarchy Enforcement**: prioritization of sources according to domain-specific authority rankings.
4. **Adversarial Tempering**: ATP validation of all high-stakes output.
5. **Confidence Calibration**: calculation and reporting of the Confidence Index.

10.5 Semantic Slicing Protocol

The Semantic Slicing Protocol (PFS—*Protocolo de Fatiamento Semântico*) decomposes complex tasks into semantically coherent units through four artisan steps:

1. **Identify**: decompose the task into semantically independent components.
2. **Classify**: assign each component to a domain and framework type.
3. **Sequence**: order components according to logical and pragmatic dependencies.
4. **Integrate**: compose the component outputs into a unified deliverable.

11 Empirical Validation

11.1 Cross-Domain Validation

Framework Injection and the Adversarial Tempering Protocol were validated across six professional domains: law (contract analysis, regulatory compliance), medicine (clinical decision support, differential diagnosis), finance (risk assessment, valuation), cybersecurity (threat analysis, vulnerability assessment), education (curriculum design, assessment development), and engineering (requirements analysis, system design). In each domain, the validation protocol was:

1. Generate output using FI with domain-specific frameworks (Forge).
2. Subject output to adversarial review by a Constitutional Auditor (Strike).

3. Incorporate validated criticisms and discard spurious attacks (Anneal).

4. Classify all findings via Sentinel; remove hallucinations from final output (Quench).

Key finding: across all six domains, tempered output contained **zero residual hallucinations**—every claim in the final output was either verified against authoritative sources or explicitly qualified with uncertainty markers. This does not mean that the Forge phase produced no hallucinations; it means that the ATP process systematically identified and eliminated them.

11.2 Confidence Index

The Confidence Index (CI) provides a quantitative measure of tempered output quality:

$$\mathrm{CI} = \frac{n_{\mathrm{OK}} + n_{\mathrm{NORMA_OK}}}{n_{\mathrm{OK}} + n_{\mathrm{NORMA_OK}} + n_{\mathrm{HALLUCINATION}}} \tag{1}$$

where n_{OK} is the count of verified-accurate findings, $n_{\mathrm{NORMA_OK}}$ is the count of normatively-compliant findings, and $n_{\mathrm{HALLUCINATION}}$ is the count of findings classified as hallucinations and removed from the final output.

Across the six-domain validation, the mean CI was **0.94** (range: 0.91–0.97), indicating that the Forge phase produced high-quality output that required relatively minor correction during the Strike-Anneal-Quench cycle. The CI measures the *quality of the initial generation*, not the quality of the final output (which, by definition, has all hallucinations removed).

12 The Ontological Framework

12.1 Digital Craftsmanship as Epistemic Praxis

The formal classification of Digital Craftsmanship follows the genus-specific-difference pattern:

- **Genus**: epistemic praxis (theory-informed, ethically-grounded, reflective practice)
- **Specific difference**: for the creation, transfer, and governance of artificial intelligence through the mechanism of framework injection

The genus *epistemic praxis* is carefully chosen. *Epistemic* (from *episteme*, knowledge) marks this as a knowledge-oriented practice, not merely a procedural one. *Praxis* (in the Aristotelian-Freirean sense) marks this as theory-informed practice oriented toward ethical ends, not merely *techne* (skill) or *poiesis* (production).

12.2 Framework Injection as Epistemic Mechanism

- **Genus**: epistemic mechanism (a process that produces epistemic effects through identifiable causal pathways)
- **Specific difference**: unidirectional transfer of complete domain-specific reasoning methods from human expert to AI agent, encoded in natural language, operating as persistent cognitive substrate

12.3 Constitutive Inherence: The Aristotelian Four Causes

The relationship between Digital Craftsmanship and Framework Injection is not mere association, composition, or dependency. It is *constitutive inherence*—a relationship best understood through Aristotle's four causes:

Table 7: Constitutive Inherence: Digital Craftsmanship and Framework Injection

Cause	**Digital Craftsmanship**	**Framework Injection**
Material	The knowledge, expertise, and ethical principles that constitute the raw material of the practice	The natural language encoding that constitutes the raw material of the mechanism
Formal	The seven-layer architecture and twelve elements that give the practice its structure	The five properties and five types that give the mechanism its structure
Efficient	The practitioner (artisan) who performs the practice	The injection event that activates the mechanism
Final	Expert-quality AI output with ethical governance	Domain-specific reasoning in the AI agent

The relationship is constitutive because FI is not merely a tool that Digital Craftsmanship *uses*; it is the mechanism through which Digital Craftsmanship *acts*. Remove FI, and Digital Craftsmanship becomes theory without mechanism. Remove Digital Craftsmanship, and FI becomes mechanism without foundation.

12.4 Five-Level Abstraction Hierarchy

The complete framework is organized in a five-level abstraction hierarchy:

Table 8: Five-Level Abstraction Hierarchy

Level	Name	Content
L5	Research Program	The Artisanal Intelligence Program (Lakatosian)
L4	Epistemic Praxis	Digital Craftsmanship
L3	Epistemic Mechanism	Framework Injection
L2	Validation Protocol	Adversarial Tempering Protocol
L1	Artifacts	Frameworks, tempered outputs, domain libraries

12.5 The Artisanal Intelligence Program as Lakatosian Research Program

Imre Lakatos's methodology of scientific research programmes [21] distinguishes between a *hard core* of irrefutable commitments and a *protective belt* of auxiliary hypotheses that can be modified without abandoning the program. A research program is *progressive* if it generates novel predictions and explains phenomena beyond its original scope; it is *degenerating* if it merely accommodates anomalies post hoc.

The **Artisanal Intelligence Program** is structured as a Lakatosian research program:

Hard core (irrefutable commitments):

1. AI output quality is proportional to the quality of the reasoning method transferred, not the quality of the command.
2. Domain-specific reasoning can be transferred from human expert to AI agent through natural language encoding.
3. Adversarial tempering produces structurally stronger output than non-adversarial generation.

Protective belt (modifiable auxiliary hypotheses):

1. The specific framework taxonomy (five types) is the optimal categorization.
2. The specific ATP phases (four) are the optimal decomposition.
3. The specific theoretical foundations (six) are the most relevant.
4. The Confidence Index formula is the optimal quality metric.

The protective belt can be modified as empirical evidence accumulates—new framework types may be discovered, ATP may benefit from additional phases, new theoretical connections may be established—without threatening the hard core commitments that define the program.

12.6 The Artisan's Paradox and Its Resolution

Definition 12.1 (The Artisan's Paradox)**.** *If Framework Injection is so effective at transferring expertise, why can't the AI agent create its own frameworks? If it can, the human artisan is eventually unnecessary. If it cannot, what limits the transfer?*

The resolution lies in the asymmetry between *operating within* a framework and *creating* a framework. Framework creation requires what Polanyi [26] calls tacit knowledge: the embodied, contextual, experiential understanding that domain experts possess but cannot fully articulate. The framework *is* the articulation of tacit knowledge—the externalization step in the SECI model. The AI agent can operate within the externalized framework with extraordinary facility (internalization), but it cannot generate the externalization from its own resources because it lacks the tacit knowledge that drives the externalization process.

The paradox is empirically unresolved in the strong sense: we cannot prove that future AI systems will never develop the capacity for autonomous framework creation. But the current resolution is clear: the artisan is necessary because the artisan possesses what the machine lacks—embodied, contextual, experiential expertise that is the *source* of frameworks, not merely their consumer.

13 Integration with Research Ecosystem

Digital Craftsmanship and Framework Injection do not exist in isolation. They are part of a research ecosystem designed to address the full spectrum of challenges in human-AI collaboration:

STEER (Structured Taxonomy for Embedding-space Exploration and Retrieval) [14]: provides sixteen algebraic operations for embedding-space retrieval, enabling precise semantic search within domain knowledge bases. STEER addresses the *retrieval* problem: how to find the right information for inclusion in context and frameworks. Integration: STEER provides the retrieval layer that feeds Context Engineering (L2 in the paradigm taxonomy), which FI then augments with reasoning methods. DOI: `10.5281/zenodo.19325724`.

IMI (Integrated Memory Infrastructure) [15]: provides cognitive memory for AI agents through graph-based, adaptive, and causal memory systems. IMI addresses the *persistence* problem: how to maintain and evolve frameworks across interactions. Integration: IMI stores and retrieves frameworks, enabling persistent framework injection across sessions. DOI: `10.5281/zenodo.19325745`.

IMI Theory [16]: provides the mathematical foundations for memory persistence using persistent homology from topological data analysis. IMI Theory addresses the *theoretical foundation* for memory: when and why certain knowledge structures persist across transformations. DOI: `10.5281/zenodo.19341740`.

SYMBIONT (Symbiotic Multi-agent Biologically-Inspired Orchestration of Networked Tasks) [17]: provides eight biologically-inspired patterns for multi-agent coordination. SYMBIONT addresses the *orchestration* problem: how to coordinate multiple framework-injected agents in complex workflows. Integration: SYMBIONT orchestrates the multi-agent ATP implementation (Semantic Artisan, Constitutional Auditor, Sentinel). DOI: `10.5281/zenodo.19325749`.

Together, these four systems and the present paper constitute the Artisanal Intelligence Program: STEER retrieves, IMI remembers, IMI Theory grounds persistence, SYMBIONT coordinates, and Digital Craftsmanship with Framework Injection provides the epistemic foundation and core mechanism.

14 Open Questions and Research Agenda

14.1 Transfer Fidelity Index

A critical open question is how to measure the *fidelity* of framework transfer—the degree to which the LLM's reasoning faithfully reflects the injected framework's methodology. We propose a Transfer Fidelity Index (TFI) as a future research direction: a metric that compares the model's reasoning trace against the framework's prescribed method, measuring alignment at each step.

14.2 Framework Composition and Interference

When multiple frameworks are injected simultaneously (e.g., legal and financial frameworks for a regulatory compliance task), how do they interact? Do they compose additively, or do they interfere? Under what conditions does composition enhance performance, and under what conditions does it degrade? This question is critical for multi-domain professional practice.

14.3 Cross-Model Portability

Frameworks are currently designed and validated for specific model families. How portable are frameworks across models—from GPT to Claude to Gemini to open-source alternatives? What framework features are model-dependent, and what features are universal? This question has significant practical implications for vendor independence.

14.4 FI vs. Fine-Tuning Boundary

Framework injection operates at inference time; fine-tuning operates at training time. Both aim to improve domain-specific performance. Under what conditions is FI sufficient,

and under what conditions is fine-tuning necessary? Is there a natural boundary—perhaps related to task complexity, domain breadth, or performance requirements—that separates the FI regime from the fine-tuning regime?

14.5 Temporal Degradation and Framework Refresh

Do frameworks degrade as models are updated? When a model receives a new training data cutoff or architecture update, do previously effective frameworks lose efficacy? If so, what maintenance regime is required to keep frameworks current?

14.6 The "Pidgin Gap" Experiment

We propose a controlled experiment comparing PE, CE, and FI across multiple domains, practitioners, and models—the "Pidgin Gap" experiment. The hypothesis: FI produces statistically significant improvements over CE, which produces statistically significant improvements over PE, across all tested domains. This experiment would provide the first rigorous, controlled evidence for the three-paradigm taxonomy.

15 Contributions

This paper makes nine specific contributions:

1. **The term "Digital Craftsmanship"** and its formal classification as *epistemic praxis*—theory-informed, ethically-grounded, reflective practice for AI creation, transfer, and governance.

2. **The term "Framework Injection"** and its formal classification as *epistemic mechanism*—the unidirectional transfer of complete domain-specific reasoning methods from human expert to AI agent.

3. **The three-paradigm taxonomy**: Command (prompt engineering) $\rightarrow$ Inform (context engineering) $\rightarrow$ Educate (framework injection), with formal inclusion relation PE $\subset$ CE $\subset$ FI.

4. **The five-type framework taxonomy**: Declarative, Procedural, Evaluative, Ethical, and Compositional frameworks, each serving a distinct epistemic function in domain reasoning.

5. **The Adversarial Tempering Protocol**: a four-phase validation mechanism (Forge $\rightarrow$ Strike $\rightarrow$ Anneal $\rightarrow$ Quench) that produces structurally stronger output through iterative adversarial stress cycles.

6. **Six theoretical connections**: the first systematic grounding of human-AI interaction methodology in Vygotsky's ZPD, Sweller's CLT, Wittgenstein's language games, Polanyi's tacit knowledge (via Nonaka & Takeuchi's SECI), Peirce's semiotics, and Greenberg's linguistic universals.

7. **The ontological framework**: constitutive inherence between Digital Craftsmanship and Framework Injection, articulated through Aristotelian four causes and organized in a five-level abstraction hierarchy.

8. **The Artisanal Intelligence Program**: classification of the research enterprise as a Lakatosian research program with identified hard core and protective belt.

9. **Empirical validation**: cross-domain validation across six professional domains with zero residual hallucinations and a mean Confidence Index of 0.94.

16 Limitations

Intellectual honesty requires explicit acknowledgment of the current limitations of this work:

1. **High barrier to entry**: creating effective domain frameworks requires genuine domain expertise. This is simultaneously a quality feature (output quality is bounded by expert quality) and an accessibility limitation (not everyone can create frameworks).

2. **No standardized benchmark**: while we have proposed the Pidgin Gap experiment and the Transfer Fidelity Index, neither has been implemented as a standardized benchmark. The comparative analysis in Section 9 relies on qualitative assessment, not standardized quantitative measurement.

3. **Cross-model portability not validated**: all empirical validation was conducted on specific model families. The degree to which frameworks transfer across model architectures and training regimes remains an open question.

4. **Framework composition not studied**: the interaction effects of simultaneously injecting multiple frameworks have not been systematically investigated.

5. **Temporal degradation not measured**: the longevity of framework effectiveness across model updates has not been measured.

6. **Artisan's Paradox empirically unresolved**: while we have provided a theoretical resolution (the tacit knowledge asymmetry), the empirical question of whether future AI systems could develop autonomous framework creation remains open.

17 Conclusion

The evolution from commanding to educating AI represents a paradigm shift in human-machine interaction—not a mere technical advance but a fundamental reorientation of the epistemic relationship between human practitioners and artificial agents.

For four years, since the public emergence of large language models in late 2022, the dominant interaction paradigm has been a form of digital pidgin: an impoverished command language that exploits a fraction of available capabilities, produces generic and frequently unreliable output, and treats the most sophisticated language-processing systems in human history as glorified command executors. Context engineering improved the situation by enriching the model's information environment, but it left the fundamental problem untouched: providing more information does not teach reasoning.

Digital Craftsmanship provides the epistemic praxis—the theory-informed, ethically-grounded, reflective practice—that elevates human-AI interaction from commanding to educating. Framework Injection provides the core mechanism—the unidirectional transfer of complete domain-specific reasoning methods that transforms generic language models into domain-specific reasoning agents. The Adversarial Tempering Protocol provides the validation discipline—the iterative stress-testing that transforms draft output into tempered output, structurally stronger and empirically reliable.

These contributions are grounded in 2,500 years of philosophical inquiry (Plato, Aristotle, Wittgenstein), supported by six independent theoretical foundations from developmental psychology to linguistic typology, validated across six professional domains, and organized into a Lakatosian research program—the Artisanal Intelligence Program—designed to evolve through progressive problem shifts while maintaining its hard core commitments.

The era of digital pidgin is ending. The era of digital craftsmanship is beginning.

Stop commanding AI. Start educating it.

References

[1] Aristotle. *Rhetoric*. c. 350 BCE. Translated by W. Rhys Roberts. Oxford University Press, 1924.

[2] Aristotle. *Poetics*. c. 350 BCE. Translated by S. H. Butcher. Dover Publications, 1951.

[3] Aristotle. *Organon* (Categories, On Interpretation, Prior Analytics, Posterior Analytics, Topics, Sophistical Refutations). c. 350 BCE. Translated by E. M. Edghill et al. Oxford University Press.

[4] J. L. Austin. *How to Do Things with Words*. Oxford University Press, Oxford, 1962.

[5] A. T. Beck. *Cognitive Therapy and the Emotional Disorders*. Penguin Books, New York, 1979.

[6] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Böckenhauer, T. Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024.

[7] B. S. Bloom. *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. David McKay Company, New York, 1956.

[8] Anthropic. Context engineering for AI agents. Technical report, Anthropic, 2025.

[9] R. Dworkin. *Law's Empire*. Harvard University Press, Cambridge, MA, 1986.

[10] H. Edelsbrunner, D. Letscher, and A. Zomorodian. Topological persistence and simplification. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science*, pages 454–463. IEEE, 2000.

[11] P. Freire. *Pedagogy of the Oppressed*. Continuum, New York, 1970. Translated by Myra Bergman Ramos.

[12] K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

[13] R. M. Gagné. *The Conditions of Learning*. Holt, Rinehart and Winston, New York, 1965.

[14] R. A. Gomes. STEER: Structured Taxonomy for Embedding-space Exploration and Retrieval — Sixteen Algebraic Operations for Semantic Vector Spaces. Zenodo, 2026. `doi:10.5281/zenodo.19325724`.

[15] R. A. Gomes. IMI: Integrated Memory Infrastructure — Cognitive Memory for AI Agents. Zenodo, 2026. `doi:10.5281/zenodo.19325745`.

[16] R. A. Gomes. IMI Theory: Persistent Homology Foundations for Cognitive Memory Systems. Zenodo, 2026. `doi:10.5281/zenodo.19341740`.

[17] R. A. Gomes. SYMBIONT: Symbiotic Multi-agent Biologically-Inspired Orchestration of Networked Tasks. Zenodo, 2026. `doi:10.5281/zenodo.19325749`.

[18] J. H. Greenberg. Some universals of grammar with particular reference to the order of meaningful elements. In J. H. Greenberg, editor, *Universals of Language*, pages 73–113. MIT Press, Cambridge, MA, 1963.

[19] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Mober, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[20] T. S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, Chicago, 1962.

[21] I. Lakatos. *The Methodology of Scientific Research Programmes: Philosophical Papers Volume 1*. Cambridge University Press, Cambridge, 1978.

[22] I. Nonaka and H. Takeuchi. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, New York, 1995.

[23] C. S. Peirce. *Collected Papers of Charles Sanders Peirce*, volumes 1–6. Harvard University Press, Cambridge, MA, 1931–1958. Edited by C. Hartshorne and P. Weiss.

[24] Plato. *Cratylus*. c. 360 BCE. Translated by B. Jowett. In *The Dialogues of Plato*, Oxford University Press, 1892.

[25] Plato. *Theaetetus*. c. 369 BCE. Translated by M. J. Levett, revised by M. Burnyeat. Hackett Publishing, 1990.

[26] M. Polanyi. *The Tacit Dimension*. Doubleday, Garden City, NY, 1966.

[27] J. R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, Cambridge, 1969.

[28] J. Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.

[29] L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1978. Edited by M. Cole, V. John-Steiner, S. Scribner, and E. Souberman.

[30] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

[31] L. Wittgenstein. *Philosophical Investigations*. Blackwell, Oxford, 1953. Translated by G. E. M. Anscombe.

[32] Q. Wu, A. Oltramari, et al. Cognitive LLMs: Integrating cognitive architectures and large language models for manufacturing decision-making. *SAGE Journals*, 2025.

[33] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[34] AGI 2025 Workshop. Cognitive design patterns for large language model interaction. *arXiv preprint arXiv:2505.07087*, 2025.